Reusing decommissioned curved objects to create new surfaces

Pierre BODET

From June 8, 2026 to July 31, 2026

GraphDeco, Inria Université Côte d'Azur

Supervised by Marzia Riso and Adrien Bousseau



Figure 1: **A prototype pumptrack module made by the original paper, being tested by a BMX rider. Picture by ©Patrick Wetzels, TU Delft.**

Introduction

The development of renewable energies has led to an increase in the number of wind turbines installed around the world. While this technology helps with the energy transition, it also raises the issue of how to manage wind turbines at the end of their lifespan, and more specifically their blades. Indeed, they are made of composite materials that allow them to be very strong, however these materials are very hard to recycle. Moreover, their length and their unusual shape also make them difficult to recycle.

The construction sector is one of the biggest consumers of natural resources and one of the largest emitters of greenhouse gases, because the production of construction materials requires a lot of energy and raw materials.

That is why, reusing wind turbine blades is a part of the circular economy, which optimises resources by extending the service life of the materials by reusing them, to limit the production of waste and the use of raw materials. Instead of being destroyed and hardly recycled, blades can be reused almost immediately for some architectural projects which need their specific geometry.

Because wind turbine blades are an aesthetically interesting object, they are often studied for reuse. They were already reused to build shelters or playgrounds. Moreover, their natural curvature can be used to create pumptracks, as the article published by Pupping et al. [1] shows both the feasibility of this approach and how it can be achieved.

Summary of the work done in the original paper

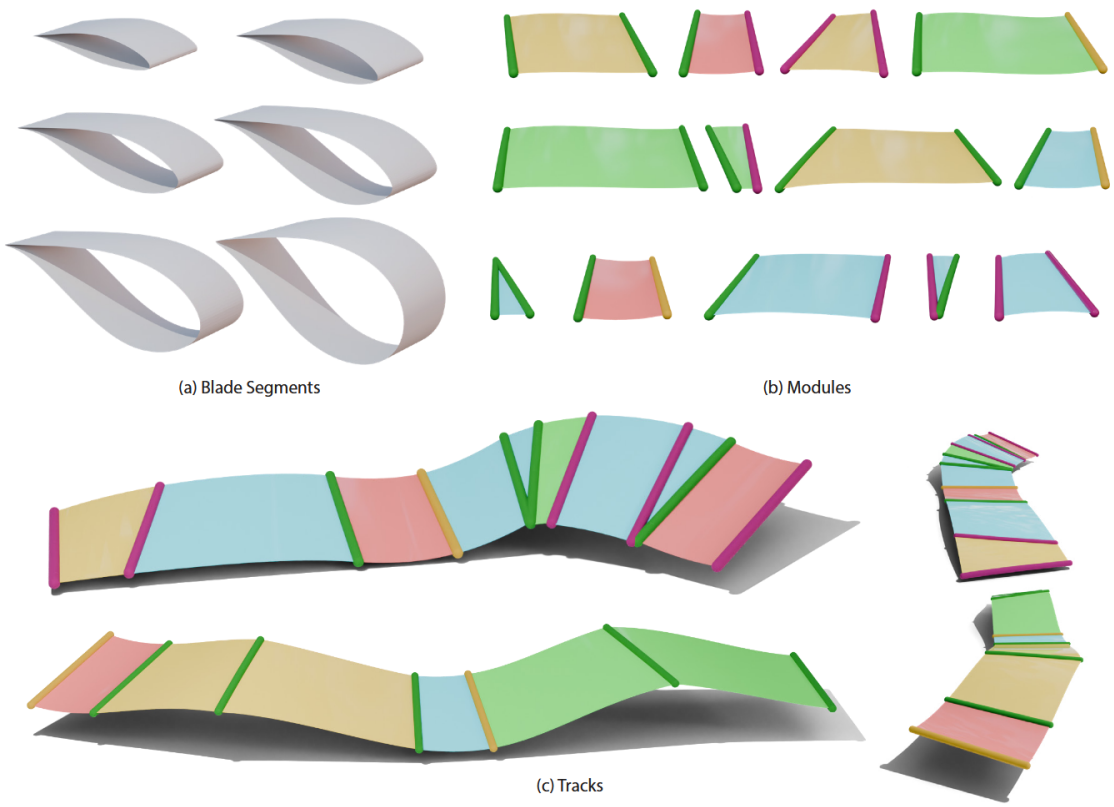


Figure 2: **Taking 6 blade segments as input (a), the algorithm extracts 36 modules and identify 6 groups of compatible boundaries (b, only a subset of modules and compatibility groups shown). The modules exhibit diverse curvature, twist, and angle between their two extremities, allowing to create tracks with corners (c, top) and rollers (c, bottom).**

To be assembled as a pumptrack, wind turbine blades are cut into several pieces. First, they are cut along the length of the blade into 1.5-meter-wide pieces (Figure 2 (a)). Then, each piece is cut into 4 smaller pieces in order to optimise the geometric continuity. They are cut horizontally always in the same way, and then cut vertically according to an algorithm so that panels can be connected in a continuous way with each other.

After that, a graph is created. Each panel is represented by two nodes in this graph: one node for its left extremity and one node for its right extremity. Each extremity of each panel is thus linked to the extremities of every panel which can be assembled with it.

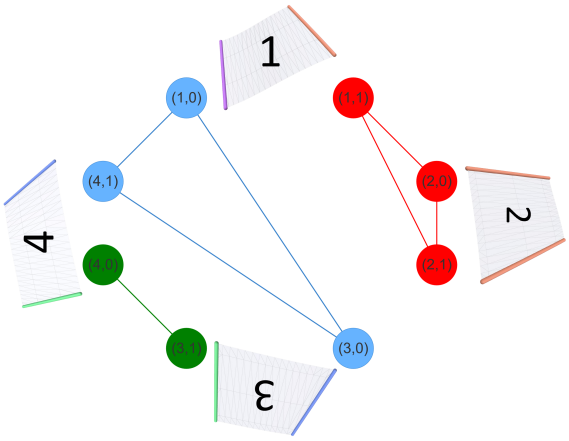


Figure 3: **A simple graph with 4 panels**

The assembly of the blade pieces was done manually. An algorithm found clusters of compatible panel extremities and coloured them with one colour per cluster. Nodes not present in any cluster were added to the closest cluster and were assigned this colour (Figure 2 (b)).

The user could select a panel with an extremity of the same colour as the available extremity of the track he was building (Figure 2 (c)).

Objectives

The objective of the internship, realised in the GraphDeco team of Inria Côte d'Azur under the supervision of Marzia Riso and Adrien Bousseau, is to automate the construction of pumptracks from the pieces of available wind turbine blades.

Instead of manually choosing one by one the panels to assemble, the user will only have to give the characteristics of the desired pumptrack to an algorithm, which will search for a track that satisfies the restrictions as much as possible. An interactive graphic interface will allow the user to easily set the parameters.

Secondly, we will implement an algorithm that, after providing it with a curve, will create a track that follows the curve as closely as possible thanks to the panels that are available.

Modelling

Principle of the search algorithm

The objective is to find a succession of panels that matches as closely as possible the desired parameters specified by the user (length, height, angle, bumps...) while satisfying construction constraints.

To find a good assembly of panels, we search for a path in the graph that matches the desired characteristics as closely as possible. However, after traversing an edge, we jump to the second node representing the panel.

An exhaustive search of every possible combination is impossible, because the number of paths is increasing exponentially with the number of panels. The algorithm therefore uses dynamic programming with a beam search strategy, which consists of keeping only a limited number of the best partial paths.

Each node of the graph represents an extremity of a panel. For each node, the algorithm stores a set of the best paths that arrive at the extremity. A path is represented not only by the list of panels used, but also by all the necessary information required to efficiently extend the track:

- the accumulated displacement along the x, y and z axes
- the cumulative transformation matrix, used to allow the algorithm to determine the exact position of the last panel
- the inclination (twist) of the track up to this panel
- the sum of the bumps already created
- the direction of the start of the track, used to calculate the final angle
- information about the last few panels, to respect the construction constraints

When a new panel is added to an existing track, all this information is updated without having to rebuild the whole track. This allows the algorithm to quickly evaluate the quality of the new track and to decide if it has to be stored or not.

To perform the search for the desired track by assembling the panels, we do not take the coloured graph, but the graph obtained before the search for the clusters. Indeed, the colours were added to help the user, but because we want to automate it, we don't need it anymore, and so we will have better data. So we can perform an adapted traversal of the graph to find the path that optimises the desired characteristics.

A pseudocode version of the algorithm used is provided below to help understand how this dynamic programming works.

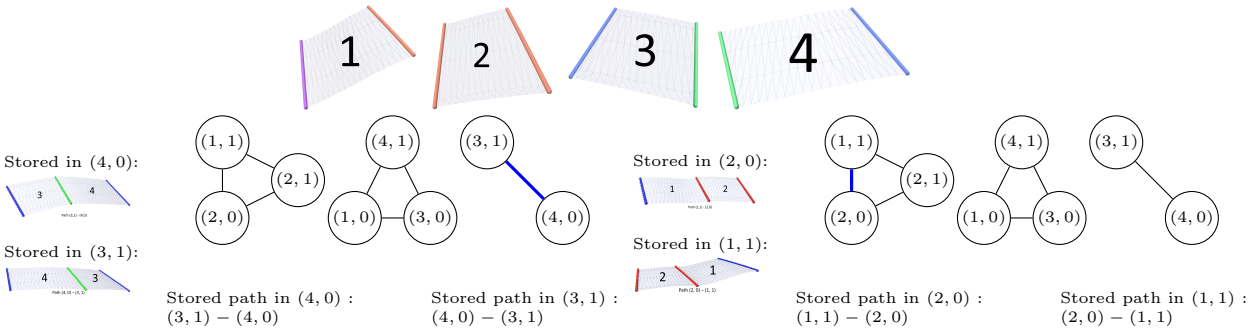
Example

Here is a small example of how the algorithm works. Figure 4 illustrates the four first steps with four panels.

At the beginning, the algorithm processes the edge $(3, 1)$-$(4, 0)$. It searches for the best paths ending at $(3, 0)$: since no such path exists, it stores only the path $(3, 1)$-$(4, 0)$ in the node $(4, 0)$. Similarly, since there are no previously computed paths ending at $(4, 1)$, it stores the path $(4, 0)$-$(3, 1)$ in the node $(3, 1)$.

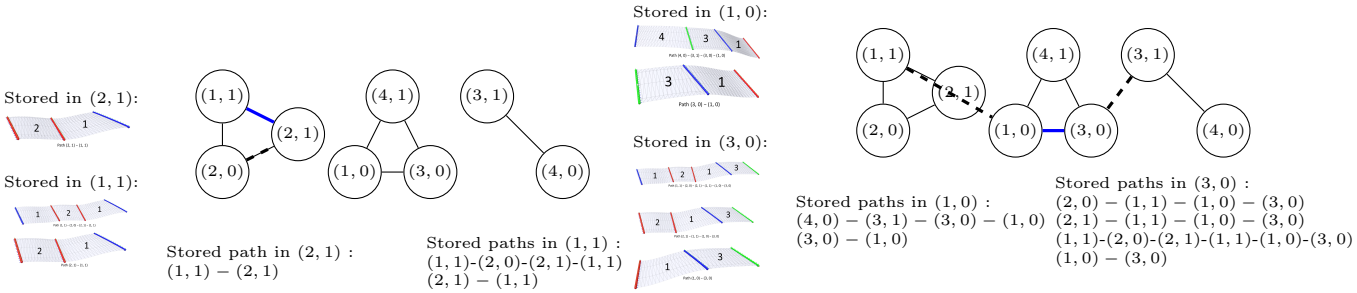
Algorithm 1 Idea of the algorithm

while the best score has changed during the last 2 iterations **do**
  for each edge (x, y) **do**
    for each previously computed path ending at the other extremity of the panel x **do**
      we calculate the characteristics of the represented track (lengths, height, angle, twist, bumps...)
      if the path is among the k best **then**
        we save it in the node y and we discard the $(k+1)$th best
        if its score is the best so far **then**
          we save everything in special variables
        end if
      end if
    end for
    for each path already computed that ending at the other extremity of the panel y **do**
      repeat the same procedure but by reversing x and y (the graph is undirected)
    end for
  end for
end while



(a) The edge $(3, 1)$–$(4, 0)$

(b) The edge $(1, 1)$–$(2, 0)$



(c) The edge $(1, 1)$–$(2, 1)$

(d) The edge $(3, 0)$–$(1, 0)$

Figure 4: **Evolution of the stored paths during the first four steps of the algorithm**

Then, the algorithm processes the edge $(1,1)$-$(2,0)$. As before, it will only store the path between the two panels in each node.

The edge $(1,1)$-$(2,1)$ is processes after. In the node $(1,0)$, no path is stored, so the path $(1,1)$-$(2,1)$ is stored in the node $(2-1)$. A path $((1,1)$-$(2,0))$ is stored in the node $(2,0)$, so the paths $(1,1)$-$(2,0)$-$(2,1)$-$(1,1)$ and $(2,1)$-$(1,1)$ are stored in the node $(1,1)$

After that, the algorithm processes the edge $(3,0)$-$(1,0)$. There is one path ending at $(3,1)$: the path $(4,0)$-$(3,1)$. It stores the paths $(4,0)$-$(3,1)$-$(3,0)$-$(1,0)$ and $(3,0)$-$(1,0)$ in the node $(1,0)$. There are three paths ending at $(1,1)$: the path $(2,0)$-$(1,1)$, the path $(2,1)$-$(1,1)$ and the path $(1,1)$-$(2,0)$-$(2,1)$-$(1,1)$. It stores the paths $(2,0)$-$(1,1)$-$(1,0)$-$(3,0)$, $(2,1)$-$(1,1)$-$(1,0)$-$(3,0)$, $(1,1)$-$(2,0)$-$(2,1)$-$(1,1)$-$(1,0)$-$(3,0)$ and $(3,0)$-$(1,0)$ in the node $(3,0)$.

Transformation matrices

Before starting the search, a precomputation step is performed. For each possible connection between two compatible panels, a transformation matrix is calculated.

This matrix is obtained by a Procrustes registration between the two curves representing the extremity of the panels. It describes exactly how to place the second panel to assemble it with the first one.

During the search, each track stores only a cumulative transformation matrix corresponding to the complete track built so far. When a new panel is added, its position is obtained by multiplying this cumulative matrix with the precomputed transition matrix.

This avoids recomputing the entire assembly at each step of the search, which would significantly slow down the algorithm.

Propagation of tracks

For each edge between two panel extremities, the algorithm retrieves all previously calculated tracks ending at the other extremity of the panel. These tracks are completed with the panel.

The new characteristics are calculated:

- the new lengths and height, by adding the actual length and height of the new panel, after applying the cumulative transformation of the current track
- the angle of the final panel, after assembling the panel
- the twist of the track by adding the panel twist.
- the new quantity of bumps, by adding the absolute value of the height

If the new track satisfies every constraint, its score is calculated and possibly added to the best stored tracks in the arriving node.

Limitation of the number of paths

Storing all possible paths would quickly lead to an explosion in the number of states stored in memory.

To limit this phenomenon, each node only stores the best k tracks (a parameter that the user can choose).

After every insertion, tracks are sorted by score and only the best ones are kept.

Without this limitation, the average number of tracks reaching each node was 50. That is why we tested with 10, 50, 100 and 200 tracks per node. The results when we only want to constrain the position of the end of the track show that 10 is already a good number. There is a small difference between the final position and the desired position but the algorithm runs much faster. If we add more characteristics to optimise, it is better to increase the number of best paths kept. The running time of the algorithm does not really change if we add more characteristics, but the accuracy increases.

Constraints applied during the search

Some constraints are checked immediately after adding a panel. If one of them is violated, the track is immediately discarded and never extended.

This strategy not only ensures that all final solutions are feasible, but also significantly speeds up the algorithm by avoiding the exploration of tracks that would become invalid anyway.

The constraints considered are:

- **Minimum and maximum height**

  At every moment, the cumulative height of the track must remain between the two values defined by the user.

  This constraint allows us, for example, to prevent the track from going under the ground or from exceeding too high.

- **Maximum twist**

  After every assembly, the normal vector of the new panel is calculated in the global coordinate system. The angle between this normal and the vertical is then measured.

  If this angle exceeds the maximum value allowed by the user, the track is rejected. This constraint avoids the creation of sections of the track that are too inclined.

- **Maximum offset along the z-axis**

At every moment, the absolute value of the cumulative displacement along the z-axis of the track must remain below the value defined by the user.

This constraint can be used by the user to have straight tracks, or to prevent the track from going beyond walls, for example.

- **Minimum length of panels**

  Panels whose projection along the main axis is smaller than 0.5 metres are not used.

  This choice is made because these panels are too short to be easily assembled and their use would complicate the construction.

- **Maximum number of consecutive panels going up or down**

  For the same reasons, when several consecutive upward or downward panels are used, supports must be placed during construction under the panels to ensure that the panels will not fall when someone will ride over them. That is why we prevent the algorithm from generating more than two consecutive panels that go up or more than two consecutive panels that go down.

Score function

Each valid track is evaluated with a scoring function.

The score corresponds to a weighted Euclidean distance between the features requested by the user and those obtained by the current path.

The factors taken into account are:

- the length along the x-axis
- the height of the track
- the length along the z-axis
- the final angle of the last panel of the track
- the quantity of bumps

The score function is:

$$w_x(l_x - l_x^*)^2 + w_h(h - h^*)^2 + w_z(l_z - l_z^*)^2 + w_\theta(\theta - \theta^*)^2 + w_b(b - b^*)^2,$$

where the values marked with an asterisk correspond to the objectives specified by the user.

The coefficients w_x, w_h, w_z, w_θ and w_b allow the user to give more weight to some characteristics. For example, if the user wants to match the desired final position of the track and simply wants a lot of bumps, the user only needs to increase w_x, w_h, w_z.

Graphical User Interface

In order to make the algorithm easily usable, a graphical user interface has been implemented with the library Polyscope. As shown in Figure 5, this interface allows the user to define all the desired characteristics of the pumptrack, and the parameters of the algorithm. Indeed, it is possible to choose the maximum number of stored tracks in each node of the graph. This is the algorithm's main parameter because it determines if the search will be fast but imprecise, or slow but precise.

The user can then specify the characteristics of the track that are the length along the x-axis, the height, the length along the z-axis, the angle of the final panel and the quantity of bumps. Each characteristic can be activated or deactivated with a checkbox. When a characteristic is activated, a value and a weight can be specified. This weight is used in the scoring function to indicate the relative importance of this characteristic. Values can also be specified by using the gizmo in the second bar in the Polyscope environment.

The interface also allows the user to define some construction constraints, such as the maximum twist or a limit on the minimum or maximum height of the track. Unlike the previous objectives, these constraints are strict, so each partial track that does not satisfy them is immediately discarded during the graph exploration.

When every parameter has been defined, the user can start the search by clicking on the Search button. After the algorithm finishes, the user will be able to view the track in the Polyscope environment, using the code written by Marzia Riso [1].

The interface also allows the user to load a curve from an OBJ file. The user can then run the greedy algorithm to build a track following this curve. The user can choose the depth parameter in the interface. Finally, the generated track can be exported as an OBJ file, so that it can be reused in modelling software.

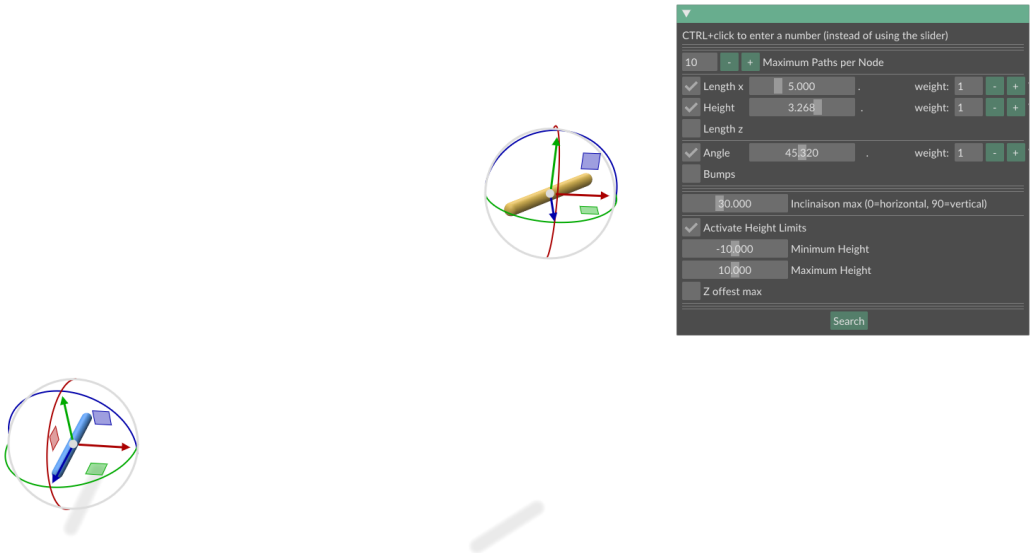


Figure 5: **The GUI, built using the Polyscope library**

Following a reference curve

In addition to the search algorithm based on global characteristics, a second approach was implemented to build a track following a reference curve provided by the user. This curve is imported from an OBJ file that contains a line. The user can easily generate the curve he wants using modelling software or a python script.

The objective is to select successively the panels whose assembly follows this curve as closely as possible.

A greedy algorithm is used. At each step, every compatible panel with the current extremity are tested. For each of them, the ending position of the panel after assembly is compared to the desired curve. The panel that minimises this distance is permanently added to the track. The distance between the track and the curve is evaluated by taking the Euclidean distance between the extremity of the panel and the position of the curve in the same plane that is normal to the direction of the curve.

To limit the decisions based only on the next panel, the user can select the number of consecutive panels that will be tested to minimise the distance and find the best one. For each panel, panels will be added recursively to explore every possibility. The algorithm will add the first panel of the best sequence, which is the closest to the given curve. Then, the algorithm will repeat the computations to add the next panel. By applying this strategy, the algorithm is less likely to become trapped in a local optimum but takes a more global view, to obtain the best possible track.

Results

The first algorithm was evaluated with different optimisation objectives and with different numbers of best paths stored in each node for the search. The results presented below allow us to analyse the influence of these parameters on the generated tracks.

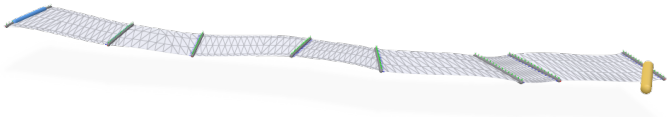
Impact of optimised characteristics

Figure 6 presents the different tracks obtained when the algorithm tries to optimise the length along the x, y and z axes.

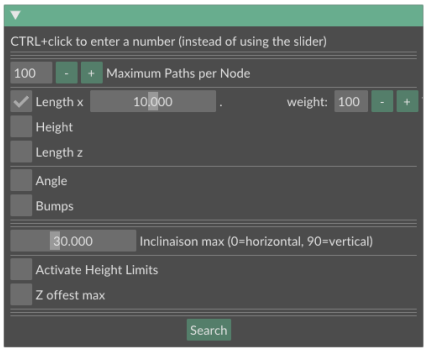
Firstly, only the length along the x-axis is taken into account. The algorithm tries only to reach this desired length, without considering the height or the displacement along the z-axis.

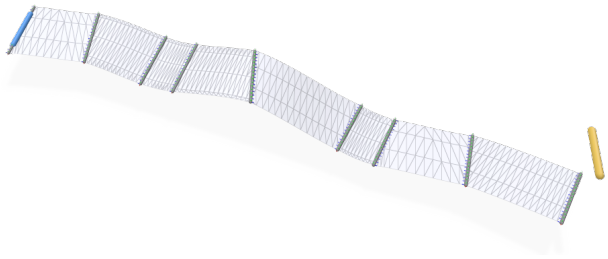
The three following examples correspond to the three possible combinations of two characteristics among the length along the x-axis, the height and the length along the z-axis. We observe that the generated track evolves to satisfy the desired objectives simultaneously.

Finally, the last example shows the track generated when the three coordinates are optimised together. The algorithm searches the best compromise between the three char-

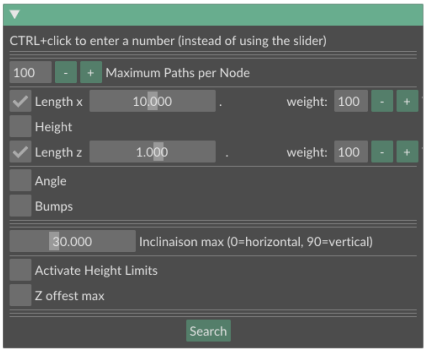


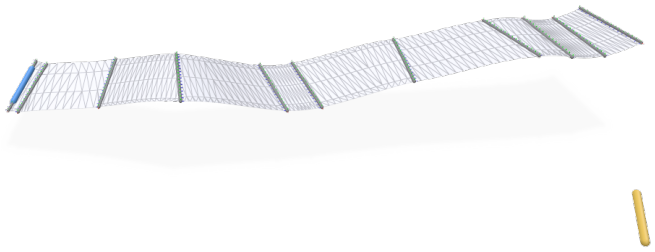
length x



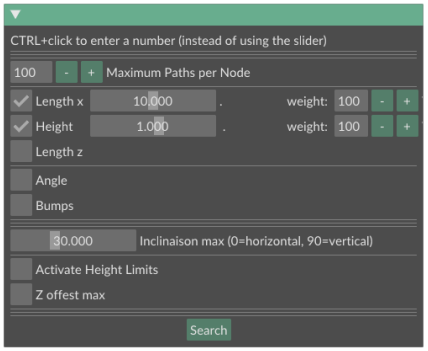


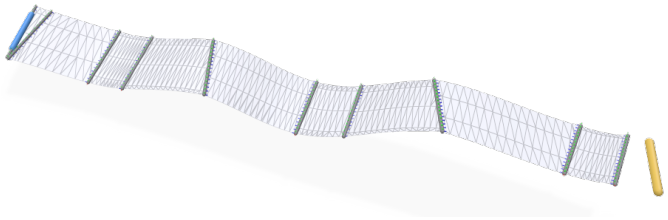
length x and length z



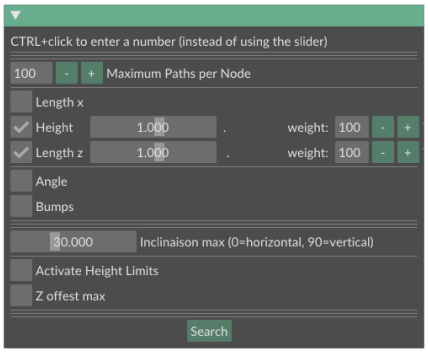


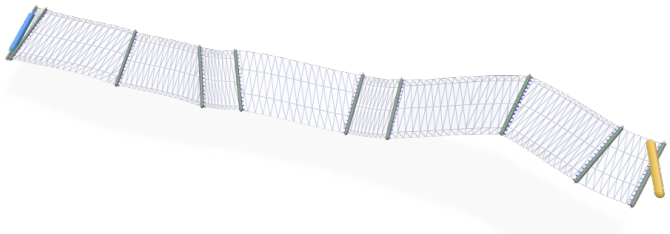
length x and height





length z and height





length x, length z and height

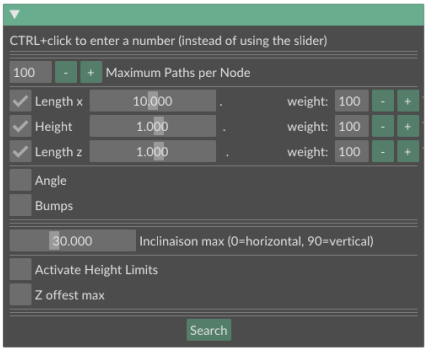


Figure 6: **Generated tracks based on the desired characteristics**

Impact of the number of stored tracks

Figure 7 compares the tracks generated when the algorithm stores the best 10, 50, 100, and 200 paths respectively in each node of the graph, when only the final position is optimised.

Keeping only the 10 best paths already allows us to obtain an appropriate solution, while greatly reducing the running time of the algorithm and the memory usage.

If we increase this number, the algorithm explores more possible tracks, which progressively improves the quality of the solution. However, the differences become smaller and smaller between storing the best 100 and 200 tracks, while the runtime keeps increasing.

These results show that a small number of stored tracks is enough when we do not optimise all the characteristics.

Table 1 shows the average runtime of the algorithm depending on the number of stored tracks. The same five position characteristics are provided to the algorithm, with a different number of stored tracks. The x length is about 10 metres

Number of stored tracks	**Average runtime (s)**
10	50.45
50	194.97
100	497.34
200	659.54

Table 1: **Average runtime in relation to the number of stored tracks**

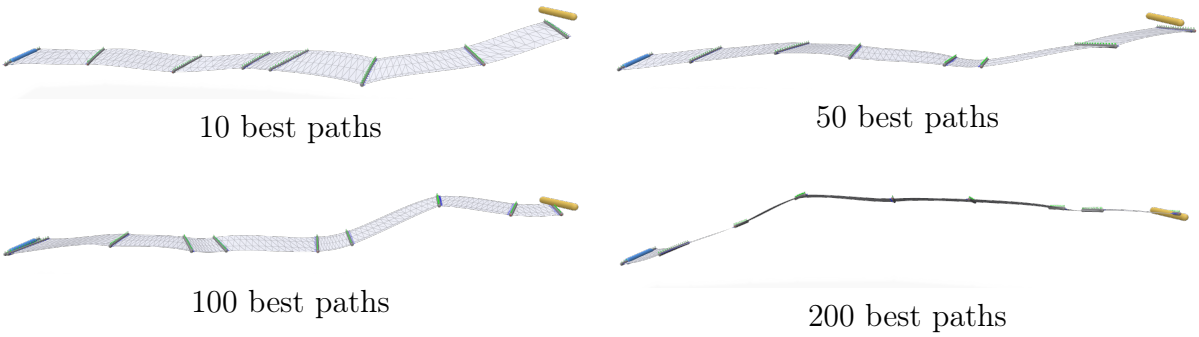


Figure 7: **Difference in accuracy based on the number of best paths stored**

Impact of additional characteristics

Figure 8 shows the results obtained when storing the best 100 paths, for different optimisation objectives in addition to the position.

The first line corresponds to an optimisation of the final position alone.

The second line adds a constraint on the final orientation of the track, with an angle of 45°.

The third line aims to obtain a track without any bumps, by asking for a total cumulative height equal to the final height of the track.

The fourth line asks for more bumps, by asking to have the total cumulative height to be equal to half of the track length along the x-axis.

These results show that when multiple characteristics are simultaneously desired, to store 100 tracks instead of 50 usually allows the algorithm to get tracks closer to the desired solution. Indeed, more intermediate tracks are explored during the search, and which increases the possibilities for improving the solution.

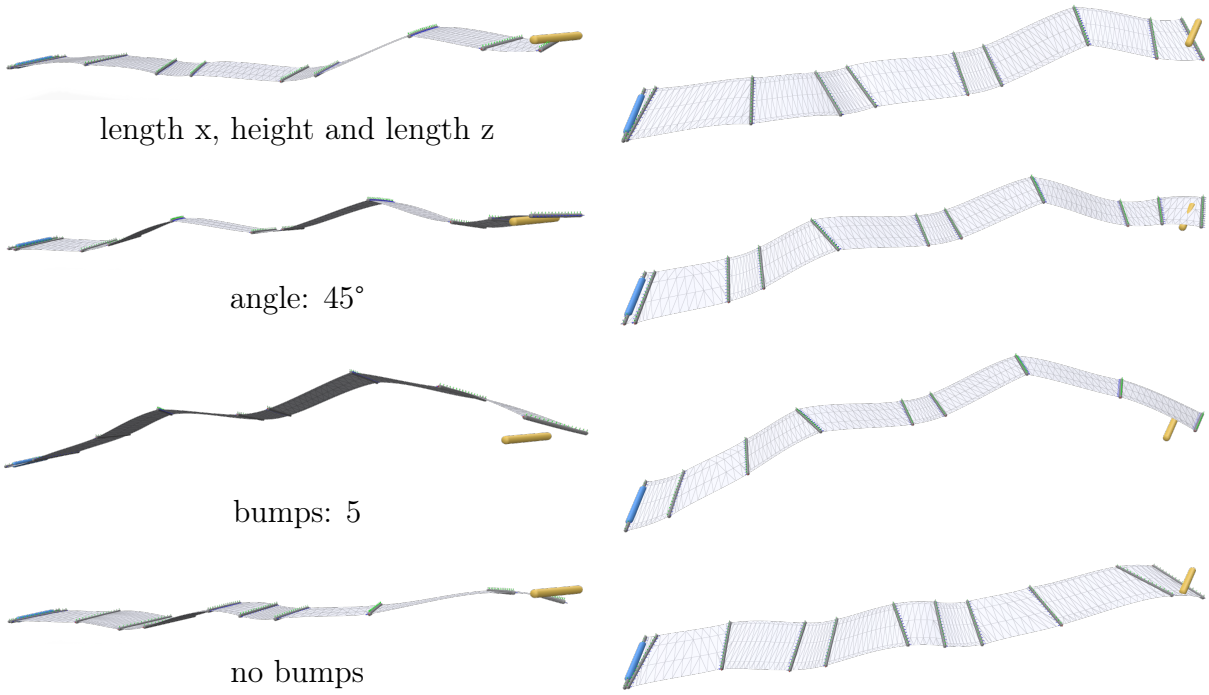


Figure 8: **Generated tracks, by storing 100 best paths, given fixed constraints on length and height, and possibly angle or bumps**

Impact of search depth on following a reference curve

Figure 9 compares the tracks obtained by the algorithm following a given curve when search depth is increased. The runtime of the algorithm is also shown.

The results show that this algorithm allows the construction of tracks close to the given curve, while respecting the constraints of assembly between the panels. The quality of the track is improved with increasing search depth, but the runtime increases very quickly because the number of explored sequences increases exponentially with the depth. However, the generated tracks are very accurate with a search depth of only three, so it is therefore unnecessary to search with a greater search depth.

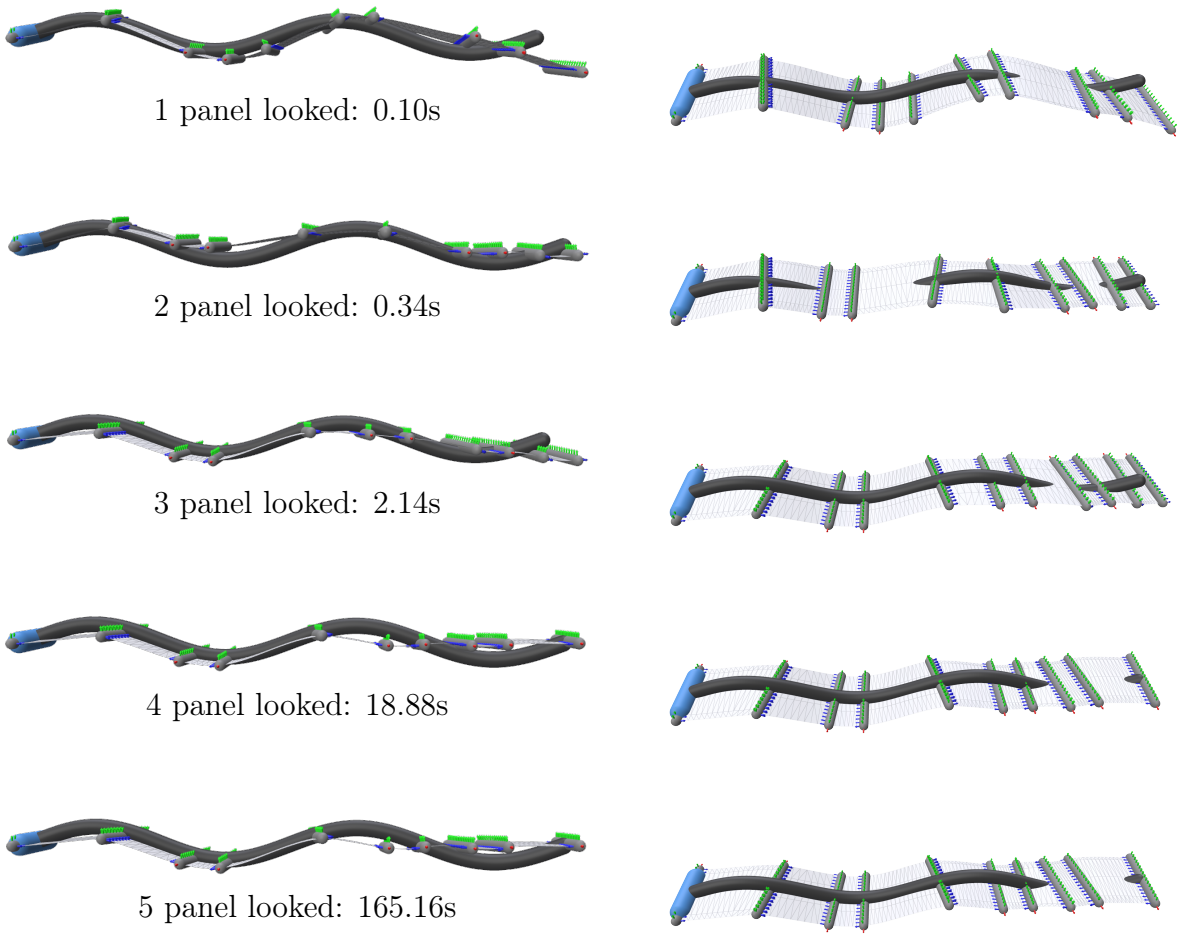


Figure 9: **Generated tracks based on a desired curve**

Limitations

Despite the obtained results, some limitations persist. Firstly, the algorithm uses a heuristic approach, that prevents it from exploring the whole solution space. By modifying the heuristic method, for example by not storing the solutions that minimise the score, but rather those that minimise each characteristic (for example, the average minimisation of each characteristic), better solutions could be obtained.

Secondly, the algorithm is based on the previously built compatibility graph. If there are problems with the connections that were calculated on the graph, these problems will also appear in the generated track. Indeed, some generated tracks contain connections between two panels that are not perfectly smooth, even though they are connected in the graph.

Finally, we still have the limitations described in the article [1], namely that a V-groove may appear when we connect two panels with non-parallel sides, and we do not try to ensure that the V-groove appears on the inner side of the track to preserve surface continuity.

Conclusion

All conducted tests show that the algorithm can automatically generate pumptracks that respect the constraints specified by the user. The generated tracks respect some construction constraints so we can easily build them in the real world.

The results also show the impact of the number of stored tracks during the search. When the number is small, the algorithm explores the solution space very quickly, but may discard tracks that would lead to better final solutions. On the other hand, a larger value allows the algorithm to explore more possibilities and improve the quality of the generated tracks, but it needs a longer runtime.

The different characteristics of the scoring function play an essential role in the final shape of the track. If we increase the value of the bumps, for example, we obtain a track that has more bumps but that cannot end exactly at the desired position. Whereas if we increase the weight associated with the position objective, the algorithm searches with tracks closer to the desired position. With the algorithm and the graphical user interface, the user can easily generate the desired tracks by adjusting every parameter.

Therefore, tests show that the use of a beam search is an efficient compromise between the quality of the solutions and the runtime of the algorithm. Although this method does not always produce the optimal track, it allows us to quickly obtain realistic tracks that satisfy almost all the constraints. This is sufficient, especially since we cannot test every possible combination to find the optimal track.

Finally, an algorithm that allows the user to generate tracks that follow a given curve, was also implemented. Unlike the previous algorithm, which uses dynamic programming to optimise some characteristics of the track, this algorithm uses a greedy approach, because many more constraints must be satisfied, as a complete reference curve is provided. Tests show that by considering only some panels to find iteratively the best panel to add to the final track, an accurate track is generated very quickly, provided that a feasible track exists.

These two algorithms thus allow the user to design the pump track he wishes to build, based on the constraints that apply. If he only needs a starting point and an ending point, he can use the first algorithm. If he wants to be more precise, he can create the desired curve in modelling software and provide it to the second algorithm. The second algorithm can also be used to modify some aspects of a track generated by the first algorithm. The user just has to export it, modify part of it and provide the modified track to the second algorithm.

References

[1] Mariana Popescu Adrien Bousseau Jelle Joustra Jesse Pupping, Marzia Riso. From blades to tracks: a case study in structural reuse of curved surfaces for circular design. *ACM Symposium on Computational Fabrication*, 2025.